

Interrogation 2 (9 décembre)

durée : 1 heure

Pour chacun des algorithmes, on justifiera avec soin :

- que l'algorithme termine (si une boucle « tant que » est utilisée).
- que l'algorithme renvoie bien le résultat demandé.

Pour les boucles « pour », on adoptera les conventions suivantes. Si $a, b \in \mathbb{Z}$, « pour i allant de a à b » signifie « pour i parcourant en croissant l'intervalle $\llbracket a, b \rrbracket$ ». Lorsque $b < a$, cet intervalle est vide, donc aucune des instructions dans la boucle « pour » n'est effectuée (ce sera par exemple le cas si on écrit « pour i allant de 1 à n », avec $n = 0$). On peut aussi écrire les algorithmes en Python, dans ce cas le code Python doit être correct, en particulier l'utilisation de « range » doit être correcte. On rappelle qu'en Python, les listes sont numérotées à partir de zéro, `len(L)` retourne la longueur d'une liste, et que `L[a:b]` retourne la liste extraite en prenant les indices entre a inclus et b exclus. L'expression `L[a:]` retourne la liste (éventuellement vide) extraite en prenant les indices à partir de a .

Exercice 1. (*Limites, équivalents*)

1. Donner des exemples montrant qu'on ne peut pas sommer les équivalents, ou composer les équivalents.
2. Peut-on passer les équivalents au logarithme ?
3. Trouver un équivalent en l'infini de $(1 + \frac{1}{n})^{n^2}$ et de $(1 - \frac{1}{n})^{n^2}$.

Exercice 2. 1. Écrire une fonction itérative prenant en entrée deux entiers m et n naturels et renvoyant l'entier $m \times n$, sans utiliser le produit ni même la somme. La seule chose que l'on pourra utiliser est une fonction « successeur » $s : \mathbb{N} \rightarrow \mathbb{N}$ qui prend en entrée un entier naturel et renvoie le nombre naturel suivant.

2. Prouver la terminaison et la correction de la fonction

Exercice 3. La fonction suivante prend une liste d'entiers. Termine-t-elle ? Que fait-elle ?

```
def f(L):  
    if L == []:  
        return 0  
    return L[0] + f(L[1:])
```

Exercice 4. La fonction suivante prend une liste d'entiers. Termine-t-elle ? Que fait-elle ?

```
def b(L):  
    n = len(L)  
    if n <= 1:  
        return L  
    for i in range(n-1):  
        if L[i] > L[i+1]:  
            temp = L[i]  
            L[i] = L[i+1]  
            L[i+1] = temp  
    return L
```